Optimize the input, not the weights

AI Methods for Science (77222) · Part I, week 4, Thursday; continues `lab_cnn.pdf`

The setup

Every gradient so far was $\partial L/\partial\theta$. Today the weights stay fixed and the gradient is taken with respect to the *pixels*: ∂(anything the net computes)$/\partial\mathbf{x}$. In PyTorch that is one line, `x.requires_grad_(True)` before the forward pass; after `.backward()` the gradient is in `x.grad`. There is no new script for this lab. You write the few lines yourself, with your AI assistant, around a network you already have.

The network. Any CNN trained with `cnn_lab.py`: `r = train(config)` returns it as `r["model"]`, in eval mode, on the CPU, the network of the best epoch. Train one at the start of the lab (the default recipe, 5 epochs on a CPU, or your E18 configuration), or reuse one you saved: `train({..., "save_weights": "cnn.pt"})` writes the weights and the architecture keys, `torch.load("cnn.pt")` reads them back, and `build_cnn(cfg)` from `cnn_lab.py` makes an empty net of the same shape to `load_state_dict` into. E28 needs a *second* net: the same call with another `"seed"` (and, if you like, a third with `"convs_per_block": 2`). Start those runs at the beginning of the lab, so they are ready when you get there.

The images. `load_cifar10()` from `mlp_lab2.py` gives the train, validation and test sets as tensors that are already standardized per channel with `MEAN` and `STD` (also in `mlp_lab2.py`); the network expects that input. It is easier to think in pixel units, so undo the standardization once, `x01 = x * STD[:, None, None] + MEAN[:, None, None]` (values in $[0, 1]$), work on `x01`, and wrap the net so that it re-standardizes inside. Then "ϵ" below means a fraction of the full range: $\epsilon = 4/255$ is four grey levels per pixel, and clipping to $[0, 1]$ keeps the result a valid image.

Compute, and the rules

Nothing here trains, except the network itself. One gradient with respect to a batch of 500 images is a fraction of a second; ten of them, seconds; the whole lab is minutes of compute on a laptop CPU. Do not edit `cnn_lab.py`. Nothing from this sheet is submitted: bring one figure that surprised you to the workshop on Monday.

The experiments

E26. Where does it look? ~15 min

The question. The net says "horse" with 97%. Which pixels is that answer made of? Does it look where you would, and, when it is wrong, what did it look at instead?

The experiment. Two maps of the same thing. *Saliency*: the absolute value of the gradient of the predicted logit with respect to each pixel, maximum over the three colours, one 32×32 map per image. *Occlusion*: no gradient; cover an 8×8 square with grey, record the probability of the predicted class, slide the square over the image with a stride of 2 and record the probability at each position, a 13×13 map. Do both for eight validation images: four the net gets right with high confidence, four it gets wrong.

The figure. three rows of eight: the images with the predicted label and its probability, the saliency maps, the occlusion maps (dark where the hidden patch hurt).

To discuss. Do the two maps agree? For the wrong answers, what was the net looking at? What does the raw gradient show that occlusion does not, and the reverse?

E27. Fool it ~25 min

The question. How far do the pixels have to move before the class flips? Can you see the change? Does the direction matter, or would any change of that size do?

The experiment. One step: $\mathbf{x}' = \mathbf{x} + \epsilon\,\mathrm{sign}(\nabla_{\mathbf{x}} L)$, where L is the cross-entropy of the true label, then clip to $[0, 1]$. Ten steps: repeat with a step of $\epsilon/4$, and after each step clip $\mathbf{x}' - \mathbf{x}$ back into $[-\epsilon, \epsilon]$ and $\mathbf{x}'$ into $[0, 1]$. The control: the same ϵ with random signs instead of the gradient's. Run all three on 500 validation images for $\epsilon \in \{1, 2, 4, 8\}/255$ and record the accuracy. Then the targeted version: replace L by the cross-entropy of a class of your choice and *descend* instead of ascending; at $\epsilon = 4/255$, what fraction of the 500 becomes that class?

The figure. accuracy against ϵ (in grey levels) for the three perturbations on one axis; and, for eight images at the ϵ where most of them flip, three rows: the image with its label and probability, the perturbation stretched to full contrast, the perturbed image with its new label and probability.

To discuss. At what ϵ does the net lose half of its accuracy, and is the perturbation visible there? Why does random noise of the same size do so little? (How much does a linear function $\mathbf{w} \cdot \mathbf{x}$ change under 3072 random signs of size ϵ, and under 3072 aligned ones?) Which target class is easiest to reach, which hardest, and does that match the confusions of E16?

E28. Does it transfer? ~10 min

The question. The perturbation was computed from one net's gradient. Does it fool a net that never saw it?

The experiment. Take the ten-step perturbations at $\epsilon = 4/255$ from E27, made against your first net. Evaluate the same perturbed images on the second net, the one trained with another seed (and on the bigger one, if you trained it). Then the reverse: attack the second net, test on the first.

The figure. a table: for each net, accuracy on the clean images, on the images attacked against itself, and on the images attacked against the other net. Optionally a curve over ϵ.

To discuss. The two nets differ only in the random seed. If an attack computed on A fools B, what do A and B have in common that neither was trained to have? Does a bigger net resist A's attack better, and does it resist its own?

E29. A cheap defence ~10 min

The question. The perturbation is high-frequency. If we blur the image before the net sees it, is the attack gone? And what happens once the attacker knows about the blur?

The experiment. Blur: a 3×3 kernel $[1, 2, 1] \otimes [1, 2, 1]/16$ applied to each colour channel (one `conv2d` with `groups=3` and reflect padding), or any blur you like. Evaluate the net on the attacked images from E27 after blurring them, and on the clean images after blurring them, at $\epsilon \in \{2, 4, 8\}/255$. Then put the blur *inside* the attacked function (the net sees `blur(x)`) and rerun the ten-step attack through it.

The figure. a table: clean, attacked, attacked then blurred, attacked through the blur, at each ϵ; and for four images the triple: attacked, blurred, with the label and probability under each.

To discuss. How much accuracy does the blur give back, and what does it cost on clean images? Once the attacker can differentiate through the defence, is anything left of it?